Practice Questions

Q.1. Solve the following recurrence relations using Master's Theorem:

a) $T(n)=2T(n/2)+n^2$

b) $T(n)=3T(n/4)+1/n$

c) $T(n) = 10T(n/3) + n^5\log^3 n$

d) $T(n)=0.5T(n/2)+n$

Q.1. Solve the following recurrence relations using the **recurrence tree method**:

$$T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + cn, \quad T(1) = \Theta(1)$$

$$T(n) = 2T\left(\frac{n}{2}\right) + n, \quad T(1) = \Theta(1)$$

$$T(n) = 4T(n/2) + n^2, \quad T(1) = O(1)$$

Q.2. Solve using the substitution/Iteration Method to solve the following recurrence relation:

$$T(n) = \begin{cases} n(n-1), & \text{if } n > 1 \\ 1, & \text{otherwise} \end{cases}$$

$$T(n) = 2T\left(\frac{n}{2}\right) + n, \quad T(1) = \Theta(1)$$

Q4. Solve the following recurrence relation using the **Iteration/Substitution, Master's and recursion tree method**:

$$T(n) = \begin{cases} 3T(n/2) + 4, & \text{if } n > 1 \\ \Theta(1), & \text{if } n = 1 \end{cases}$$

$$T(n) = \begin{cases} 5T(n/5) + cn, & \text{if } n > 1 \\ \Theta(1), & \text{if } n = 1 \end{cases}$$

Fractional Knapsack Problem:

Q.5. Given the following items-

Item	Weight (W)	Value (V)
A	5	100
B	7	150
C	2	50
D	10	200
E	4	75

Knapsack capacity = 15

- Calculate the profit/weight ratio for each item.
- Arrange items in decreasing order of ratio.
- Select items using the greedy method.
- Find the maximum profit.

Q.6. A thief enters a shop with a knapsack that can hold at most **50 kg**. He finds the following items:

Item	**Value (₹)**	**Weight (Kg)**
1	60	10
2	100	20
3	120	30

Using the **fractional knapsack approach**, determine the maximum profit achievable. Show the selection of items and fractions clearly.

Activity selection Problem:

Q.7. Select the maximum number of non-overlapping activities from given start and finish times. Show order in which the activities are selected.

	A1	A2	A3	A4	A5	A6	A7	A8	A9	A10
Start	0	3	0	4	7	8	9	7	8	13
Finish	6	5	4	6	11	10	11	8	16	16

Q.8. Select the maximum number of non-overlapping activities from given start and finish times. Show order in which the activities are selected. Also mention which activities selected.

ACTIVITY	START TIME	END TIME
A1	1	3
A2	2	5
A3	3	4
A4	4	7
A5	7	10
A6	8	9

Q.9. Find the optimal solution for the 0/1 knapsack problem making use of dynamic programming approach. Consider-

$$n = 4, w = 5 \text{ kg}$$

(w1, w2, w3, w4) = (2, 3, 4, 5)
(b1, b2, b3, b4) = (3, 4, 5, 6)

Sol.

Item1 and Item2

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

T-Table

Q.10. A thief has a knapsack that can carry a maximum weight of W = 8 units. He has 4 items, each with a specific weight and profit as shown below:

Item	A	B	C	D
Profit	2	3	4	1
Weight	3	4	5	6

Using the 0/1 Knapsack Dynamic Programming approach, do the following:

- Construct the DP table with rows as items (0 to 4) and columns as weights (0 to 8).
- Show the table filled step-by-step for each item.
- Identify the maximum obtainable profit.
- Using backtracking, determine which items are included in the optimal solution.

Sol.

i \ w	0	1	2	3	4	5	6	7	8
0 items	0	0	0	0	0	0	0	0	0
1 (A: 3,2)	0	0	0	2	2	2	2	2	2
2 (B: 4,3)	0	0	0	2	3	3	3	5	5
3 (C: 5,4)	0	0	0	2	3	4	4	5	6
4 (D: 6,1)	0	0	0	2	3	4	4	5	6

- Maximum Profit: 6
- Selected Items: A and C

Q.11. 0/1 Knapsack Problem

Consider a knapsack with a capacity of **50 units**. You have the following items:

Item	Value (₹)	Weight (Kg)
1	60	10
2	100	20
3	120	30

Solve the problem using the **0/1 Knapsack (Dynamic Programming)** method and find the maximum value obtainable.

Q.12. Longest Common Subsequence (LCS)

Find the length of the **Longest Common Subsequence** between the strings:

X="ABCBDAB", Y="BDCAB"

Also, write down the LCS obtained.

Q.13. We have two strings ***X=BACDB*** and ***Y=BDCB*** to find the longest common subsequence.

X = BDCB, Y = BACDB

Sol.

		0	1	2	3	4
			B	D	C	B
0		0	0	0	0	0
1	B	0	1	1	1	1
2	A	0	1	1	1	1
3	C	0	1	1	2	2
4	D	0	1	2	2	2
5	B	0	1	2	2	3

The longest common subsequence of sequences X and Y is BCB.

Q.14. Find time and space complexity of the given algorithms.

(a) Algorithm LinearSearch(A, n, x)

```
for i ← 1 to n do
    if A[i] = x then
        return i
```

return -1

(b) Algorithm MatrixAdd(A, B, n)

for i ← 1 to n do

for j ← 1 to n do

C[i][j] ← A[i][j] + B[i][j]

return C

(c) Algorithm TestAlgo(n)

i ← 1

while i ≤ n do

print(i)

i ← i * 2

Recursive Algorithm

(d) Algorithm F(n)

if n <= 1

return 1

for i = 1 to n

perform constant-time operation

F(n/2)

F(n/2)

Question:

- Form the recurrence relation.
- Solve the recurrence and determine the time complexity.

Divide-and-Conquer Algorithm

(e) Algorithm Solve(n)

if n <= 1

return

Solve(n/2)

Solve(n/2)

```
for i = 1 to n
    perform constant-time operation
```

Question:
Find and solve the recurrence relation and determine the time complexity.

Unequal Recursive Subproblems

(f)
```
Algorithm Solve(n)
    if n <= 1
        return

    Solve(n/2)
    Solve(n/4)

    for i = 1 to n
        perform constant-time operation
```

Question:

- Form the recurrence:

$$T(n)=T(n/2)+T(n/4)+\Theta(n)$$

- Solve the recurrence.
- Determine the asymptotic complexity.

(g)

```
Algorithm A(n)
    if n <= 1
        return
    A(n/2)
    for i = 1 to n
        for j = 1 to n
            perform constant-time operation
```

Find the recurrence relation and solve it.

(h).

```
Algorithm A(n)
    if n <= 1
        return
    A(n-1)
    for i = 1 to n
        perform constant-time operation
```

Find the recurrence relation and determine the exact/asymptotic time complexity.

(i)

```
Algorithm A(n)
    if n <= 1
        return

    A(n/2)
    A(n/2)

    for i = 1 to n
        for j = 1 to i
            perform constant-time operation
```

Determine the recurrence and solve it.

(j)

```
Algorithm A(n)
    if n <= 1
        return

    A(n-1)

    for i = 1 to n
        perform constant-time operation
```

Question:
Write the recurrence and solve:

$$T(n)=T(n-1)+n$$

Then determine the complexity.

(k) Consider:

```
Algorithm A(n)
  if n <= 1
    return

  A(n/3)
  A(2n/3)

  for i = 1 to n
    perform constant-time operation
```

Question:

- Form the recurrence.
- Draw the recursion tree.
- Determine the height of the tree.
- Calculate the total work at each level.
- Derive the final time complexity.

(l)

```
Algorithm Solve(n)

  if n <= 1
    return

  if n is even
    Solve(n/2)
  else
    Solve(n-1)
```

```
    for i = 1 to n
        perform constant-time operation
```

Questions to ask:

1. Identify the **base condition**.
2. Write the recurrence relation for the **even case**.
3. Write the recurrence relation for the **odd case**.
4. Solve the recurrence for each case.
5. Determine the **overall time complexity**.

(m) Algorithm Divide(n)

```
    if n <= 1
        return 1

    if n > 10
        x = Divide(n/2)
        y = Divide(n/2)
    else
        x = Divide(n-1)

    for i = 1 to n
        perform constant-time operation

    return x
```